

TRACKING THE RISE OF AI/ML IN OPEN-SOURCE DEVELOPMENT AT BIG-DATA SCALE DATASET: GITHUB ARCHIVE (GH ARCHIVE)

AUTHORS

Arushi Nirmal & Yixiu Liu ·
CS 131 Final Project ·

INTRODUCTION

AI/ML has reshaped software over the last five years. GitHub — the world's largest open-source platform — records every push, PR, issue, star and fork publicly. By measuring this activity at scale we can see, with real data, how fast the AI/ML ecosystem is actually growing versus software as a whole.

RESEARCH QUESITON

Are AI/ML repositories growing faster than general software repositories from 2021–2026, measured by event volume, event mix (push / PR / issue / star / fork), and new-repo creation?

DATASET

GH Archive — an hourly archive of the public GitHub event timeline. Each hour is one gzip-compressed newline-delimited JSON file (<https://data.gharchive.org/YYYY-MM-DD-H.json.gz>), one JSON object per event (PushEvent, PullRequestEvent, IssuesEvent, ForkEvent, WatchEvent, ...), each carrying type, actor, repo, payload, created_at.

PHASE 1 - PROFILING

EXPLANATION

```
du / wc — How big? How many rows?
• time ls *.json.gz | wc -l (0.006 s)
• Output: 552 files
• time du -ch *.json.gz | tail -1 (0.006 s)
• Output: 35 GB compressed
• time zcat *.json.gz | wc -c (38m 29.020s)
• Output: 273,494,892,306 bytes, or approximately 273.49 GB uncompressed
• time zcat *.json.gz | wc -l (38m 37.140s)
• Output: 85,339,282 GitHub events
• Ten-core parallel byte count (13m 26.302s)
• Output: the same 273,494,892,306 bytes, with a 2.9x speedup.
```

```
head / tail / cut / column — Peek at the schema
• time zcat 2024-01-15-15.json.gz | head -1 | jq . (0.013 s)
• Identified the main fields: id, type, actor, repo, payload, public, and created_at.
• time zcat 2024-01-15-15.json.gz | tail -1 | jq -c ... (9.290 s)
• tail was slower because gzip had to decompress the entire hourly file.
• time ... | cut -d',' -f2 | sort | uniq -c | sort -nr (19.560 s)
• Extracted and counted event types from one hourly file.
• time ... | sort | uniq -c | sort -nr | column -t (17.718 s)
• Displayed the event counts in an aligned table.
```

```
sort | uniq -c | sort -nr — Top categories and cardinality
◦ Event-type distribution command (30m 4.864s)PushEvent: 52,244,295, or 61.2%
◦ CreateEvent: 10,597,330
◦ PullRequestEvent: 6,005,006
• Distinct-actor count command (30m 14.693s)
• Output: 8,268,278 distinct actors
• Distinct-repository count command (29m 30.237s)
• Output: 13,206,596 distinct repositories.
```

```
awk — Numeric aggregate
◦ Average commits-per-push command (29m 30.442s)Total commits: 159,502,273
◦ Push events: 52,244,295
◦ Average: 3.053 commits per push.
```

```
grep -c — Filtering counts
• AI/ML keyword filtering command (17m 43.542s)
• Output: 787,912 matching events, approximately 0.92% of all activity.
• PullRequestEvent filtering command (12m 29.191s)
• Output: 6,005,006 events, exactly matching the independent jq count.
```

PHASE 2 — BREAKING

EXPLANATION

8 Hours data is big enough to let Excel and pandas braked

In-memory tools fail BEFORE the analysis even starts — Excel silently throws away 35% of the rows, pandas dies just reading the bytes — because both must hold everything at once.

Streaming (constant memory) and distributed (bounded memory per core) tools finish the identical aggregation in minutes/seconds.

This spreadsheet may look different.

Here's a list of what changed when you opened it in Numbers.

Your data has 1,612,733 rows. Tables in Numbers can't support more than 1,000,000 rows. Some content was not imported.
Sheet: Sheet 1 (Table: excel_rowcap)

```
ishigaki-cs6@spark-833c:~/final_project/2 breaking$ cd ~/final_project
( ulimit -v 8000000 # cap THIS subshell at ~8 GB virtual memory
  venv/bin/python -c "import pandas as pd
df = pd.read_json('data/day.json', lines=True)
print(df.shape)"
Traceback (most recent call last):
  File "<string>", line 2, in <module>
  File "/home/ishigaki-cs6/final_project/.venv/lib/python3.12/site-packages/pandas/io/json/_json.py", line 829, in read_json
    json_reader = JsonReader(
                  ~~~~~
  File "/home/ishigaki-cs6/final_project/.venv/lib/python3.12/site-packages/pandas/io/json/_json.py", line 936, in __init__
    self.data = data.read(
                  ~~~~~
MemoryError
```

CONCLUSION

AI/ML activity grew 2.35× from 2021 to 2026 vs 1.28× for general software, peaking at 2.26% in 2023-Q2 alongside GPT-4 — yet AI/ML is still under 1% of all public GitHub events. The durable change isn't volume but attention: AI/ML repos are starred 4.4× and forked 3.1× more than general repos while being pushed to less, in every usable quarter. The AI boom shows up on GitHub as an attention boom first, and a code boom second.

PHASE 3— SCALING

Experiment setup

- Processed 85.3 million GitHub events from 552 gzip JSON files
- Raw data remained in Google Cloud Storage and was read directly through gs://
- Used the same PySpark job for every experiment
- Tested three compute configurations:
 - Single-node: 4 shared vCPUs
 - 2 workers: 8 worker vCPUs
 - 4 workers: 16 worker vCPUs
- Job included schema definition, filtering, classification, groupBy, joins, window functions, and caching

PySpark Data Engineering Workflow on Dataproc

From raw GH Archive JSON in GCS to monthly analytics with growth (same job, any cluster size)



Cluster	Worker vCPUs	Job RUNNING→DONE	Speedup	Efficiency	End-to-end `time`
single-node	4 (shared)	2617 s (43m37s)	1.00x	—	43m49.6s
2 workers	8	1048 s (17m28s)	2.50x	1.25	17m37.2s
4 workers	16	621 s (10m21s)	4.21x	1.05	10m38.5s

TOOLS USED

Phase 1 — Profiling (CLI, streaming): zcat, wc -c/wc -l, jq, grep -c, cut, awk, sort, uniq -c, column -t, parallelised with xargs -P 10, every command timed with time.

Phase 2 — Breaking (in-memory): Excel / Numbers row cap, pandas read_json under ulimit -v, peak RSS from /usr/bin/time -v.

Phase 3 — Scaling (distributed): PySpark on GCP Dataproc, raw data read straight from a GCS bucket via gs://..., clusters managed with gcloud dataproc.

PHASE 4— ANALYSIS

EXPLANATION

Loaded Phase 3's aggregated CSVs (43 KB from 273.5 GB) into pandas and rejected 3 of 23 sampled days first — 21Q4 had only 32% of a median day's events, 26Q2/Q3 lost stars and PRs upstream — so the ~68.65% in the growth CSV is an archive artifact, not a real collapse.

AI/ML activity grew 2.35× from 2021 to 2026 vs 1.28× for general software (18.7%/yr vs 5.1%/yr) — 3.7× faster, but still under 1% of all public GitHub events. Share peaked at 2.26% in 2023-Q2, the GPT-4 + AutoGPT quarter, when distinct AI-repo actors jumped 3,734 → 19,977 in three months. Most left; a 1.56× permanent lift remains.

The durable difference isn't volume — it's attention. AI/ML repos are starred 4.4× and forked 3.1× more than general repos while being pushed to less, in every one of the 20 usable quarters.

The AI boom shows up on GitHub as an attention boom first, and a code boom second.

